Programación de Sistemas y Dispositivos

Grado en Ingeniería Informática

Scheduling (Planificador de tareas)

Prof. Jose Antonio Huerga

Conceptos e implementación

Objetivos:

- Conocer los principales conceptos y políticas de scheduling (*Task Scheduling*).
- Técnica *Simple cyclic*, *Timed cyclic*, planificador *cooperativo* y *time slicing* (*Round-Robin*).
- Métodos de scheduling y diferencias entre sistemas *preemptive* y *non-preemptive*.
- Explicar la necesidad de asignar prioridades a las tareas (*Task priorities*).
- Análisis del bloque de control de tareas (*TCB*).
- Consideraciones sobre compartición de código seguro (*secure code sharing*) en RTOS.
- Conocer la *impredecibilidad* en run-time.

Los conceptos son aplicados a sistemas mono core o single CPU, como los que suelen verse en la mayoría de microcontroladores del mercado.

Conceptos e implementación

Simple Cyclic

Se trata de una de las soluciones más simples. En este caso, el planificador o scheduler organiza las Tasks formando una cola o queue. El *scheduler* asigna el control total de la CPU a la primera tarea de la cola, con la diferencia de que no se libera el uso de la CPU hasta que la tarea complete todo el proceso que se le ha asignado. Esto se denomina *Run to completion.*

Una vez que la primera tarea de la cola (*Head of the Queue*) completa sus procesos, se libera el uso de la CPU y se asigna a la siguiente tarea de la cola. Este cambio de tarea (*Task context* o *Task switching*) es controlado por el *scheduler/dispatcher*.

Como hemos dicho antes, el *scheduler* no controla directamente las actividades mismas de cada tarea, siendo la propia tarea la encargada de liberar el uso de CPU. Por tanto, la primera tarea completa sus actividades, libera la CPU y el *scheduler* asigna a una nueva tarea el tiempo de CPU.

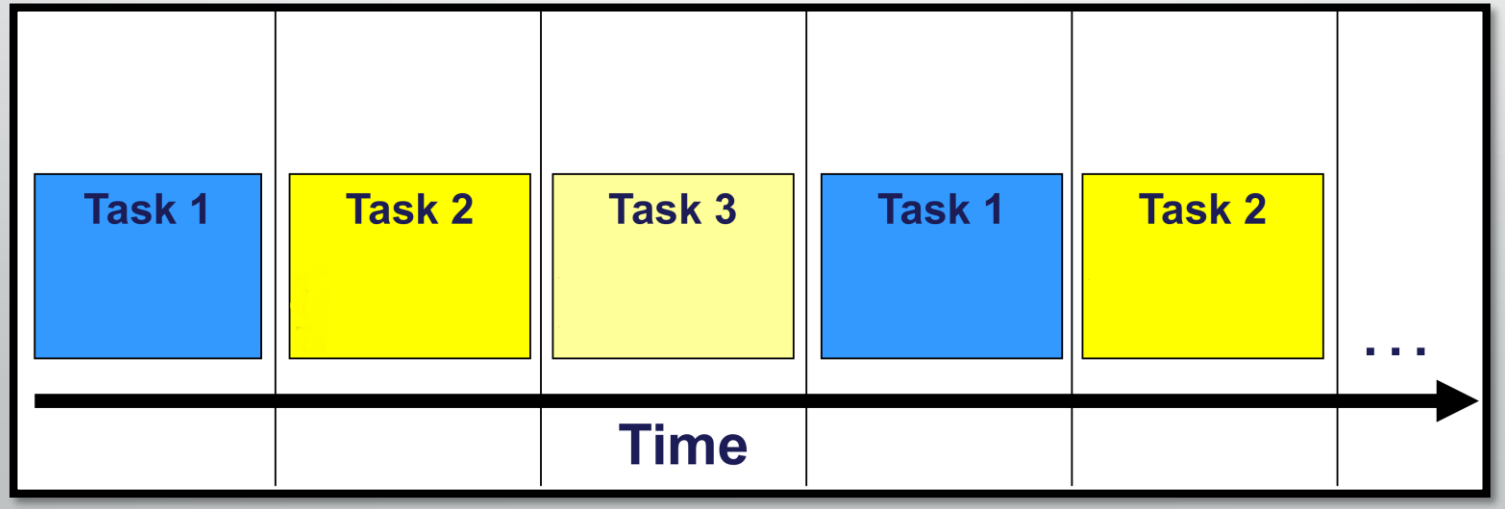
Este proceso se repite hasta completar todas las tareas de la cola. El *scheduler* trabaja como una FIFO (*First-In First-Out*).

Conceptos e implementación

Simple Cyclic

Este tipo de planificación se utiliza en sistemas en los que:

- Las tareas no son críticas y no tiene tanta importancia como afectan al sistema.
- No son críticas en el tiempo (*Time Critital Tasks*)
- No importa que las tareas se ejecuten hasta completar sus actividades (Run to completion).
- No se precisa de una tarea IDLE.



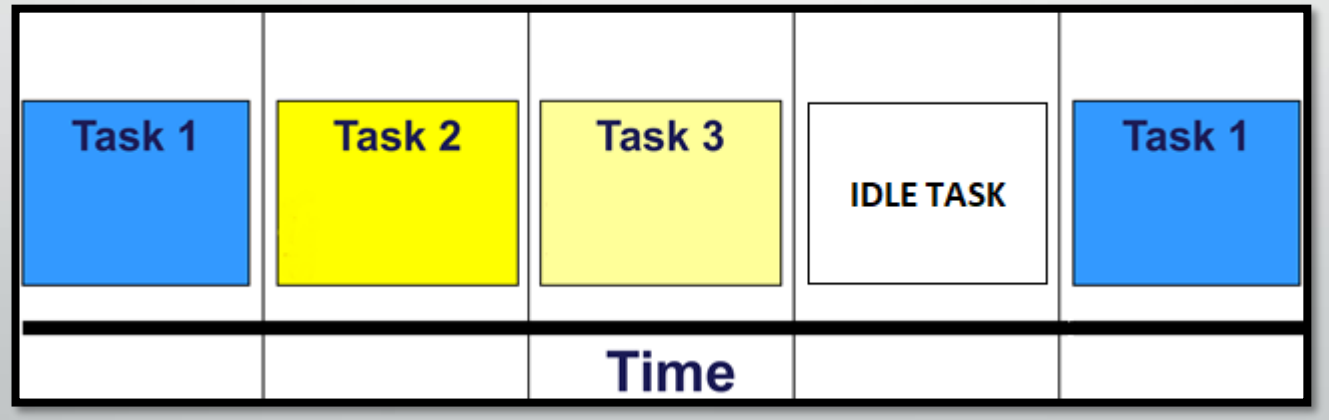
Conceptos e implementación

Timed Cyclic

Este tipo de *scheduling* es una modificación del anterior, para adaptarse a las necesidades de lo que suelen requerir la ejecución de procesos en un sistema embebido (a nivel básico). El problema general, es que el conjunto de tareas debe poder ejecutarse a intervalos de tiempo que deben estar predeterminados.

Para ello usamos una tarea extra: *Idle Task*.

IDLE Task se ejecutará al final de la cola o set de Tareas. Debido a las variaciones de timing de cada tarea, la duración de la tarea idle podrá variar. De esta manera, el periodo de ejecución del conjunto total de tareas se realiza a intervalos determinados y constantes en el tiempo.



Conceptos e implementación

Timed Cyclic. Limitaciones

- Si una tarea falla o se queda estancada, será imposible liberar el uso de la CPU y por tanto el sistema se quedará estancado (*Halt state*).
- Añadir una nueva tarea al sistema implicará que será colocada al final de la cola. Largos delays debido a otras tareas implicarán que la nueva tarea deba ejecutarse más tarde. Por tanto, los tiempos de reacción son lentos ante peticiones de ejecución de nuevas tareas.
- El sistema es gravemente afectado por tareas de larga duración. Por tanto, aunque las nuevas tareas tomen poco tiempo de CPU, deberán esperar por en espacio o slot para poder ejecutarse.
- El punto anterior puede solucionarse permitiendo que las tareas de larga duración liberen deliberadamente el uso de la CPU, para que otras tareas (de menor duración), puedan ejecutarse. Este proceso se denomina *Interleaving.*

Interleaving entre Task1 y Task2

Conceptos e implementación

Cooperative Scheduling

Este tipo de scheduling se basa en la solución al anterior punto de las limitaciones del Time Cyclic Scheduling.

Se trata de ofrecer al usuario la posibilidad de aplicar interleaving. Cuando se disponde de esta funcionalidad y se aplica al sistema, estaremos trabajando de manera cooperativa.

De esta manera, las tareas cooperan con otras con el fin de mejorar la respuesta ante eventos o bien para cumplir con el propósito del sistema para cumplir sus objetivos con mayor eficacia. Para poder desarrollar esta funcionalidad, el RTOS debe disponer de una API o función de liberación de CPU o yielding (rtos_yield). La decisión de esta liberación viene de la mano de la tarea, no del scheduler.

Cooperative Scheduling

Conceptos e implementación

Cooperative Scheduling. Limitaciones

Para mejorar el comportamiento del sistema y no depender de una función de *yielding* o liberación de CPU, se pueden implementar metodologías para usar tareas con tiempos de ejecución acotados. Es decir, a cada tarea se la asigna un tiempo máximo de ejecución (*Time Quanta*). Esto se conoce como *Time Slicing* (*Round-Robin Scheduling*).

Conceptos e implementación

Round-Robin Scheduling

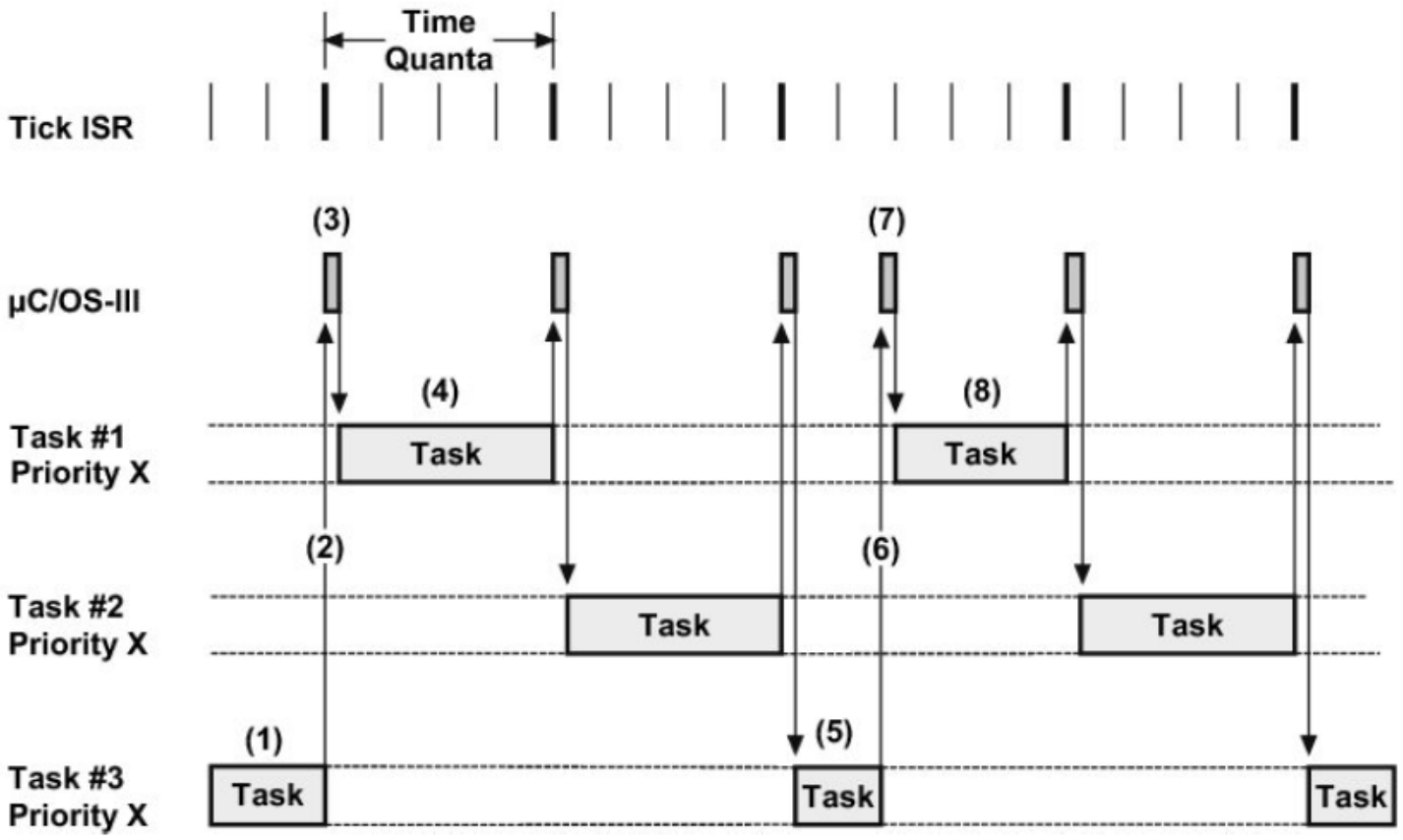


Figure - Round Robin Scheduling

En *uC/OS-III*, cuando dos o más Tasks tienen la misma prioridad, el propio planificador asigna un tiempo de ejecución o time slot a cada una de ellas. Esta tarea será ejecutada durante el llamado *Time Quanta.*

Este tiempo es configurable por el usuario. A su vez, se permite asignar un Time-Quanta particular a cada tarea, lo que ofrece flexibilidad.

Conceptos e implementación

Round-Robin Scheduling. APIs de configuración.

```
OSSchedRoundRobinCfg()
```

Permite modificar en run-time el time quanta configurado por defecto. Además, ofrece la posibilidad de habilitar o deshabilitar la función Round-Robin.

```
OSTaskTimeQuantaSet()
```

Permite modificar en run-time el time quanta particular de cada Task. Este slot de tiempos se mide en Ticks. Una tarea puede tener un time quanta de 1 tick, otra de 12 ticks, etc. El valor de time-quanta es asignado por defecto durante la creación de la propia Task, mediante la función `OSTaskCreate()`.

Conceptos e implementación

Round-Robin Scheduling

```
#define OS_CFG_SCHED_ROUND_ROBIN_EN     0u    /* Include code for Round-Robin scheduling
```

En la diapositiva anterior se han presentado las funciones que permiten modificar el time-quanta, y habilitar/deshabilitar Round-Robin en run-time. Sin embargo, para poder incluir el código que permite ejecutar Round-Robin scheduling, es necesario indicarlo como configuración del Kernel, en tiempo de precompilación. Si se desea, es decisión del usuario poder deshabilitarlo en run-time (decisión que solo es posible si se ha incluido el código necesario previamente)

Este flag de configuración lo encontramos en `os_cfg.h`

ROM Variable	Data Type	Value
OSDbg_SchedRoundRobinEn	CPU_INT08U	OS_CFG_SCHED_ROUND_ROBIN_EN

Conceptos e implementación

Round-Robin Scheduling

El planificador de tareas en modalidad *Round-Robin* asigna unos tiempos de trabajo (*time quanta*) a cada una de las *Tasks*. Basándonos en la planificación cooperativa como base, cuando una tarea se encuentra ejecutándose y finaliza su *time quanta*, el planificador asigna el tiempo de *CPU* a la siguiente tarea (que en el contexto más sencillo se ejecutan bajo la lógica *First-In First-Out*). La unidad de tiempos del *time quanta* es medida en *Ticks* del *RTOS*.

Por tanto, en un *RTOS cooperativo* (*non-preemptive* multitasking), las *Tasks* ceden de manera voluntaria el control al *Kernel* para que se produzca un switch a otra *Task*. Este proceso de ceder el control puede ser, o bien de manera implícita (la propia tarea finaliza) o de manera explícita mediante una llamada al *Kernel* (mediante una función de *yielding*).

Es responsabilidad del programador asegurar que las tareas puedan ceder regularmente el control de la *CPU* al *Kernel*, especialmente en tareas con porciones de código extensas, bucles de espera activa, etc). Cuando se cede el control de la *CPU*, el *Kernel* pasará a ejecutar la siguiente tarea de la lista, o bien la siguiente tarea más prioritaria, en función de la complejidad y modo de trabajo del *RTOS*.

Conceptos e implementación

Round-Robin Scheduling. Resumen de funciones de uC/OS III

OSSchedRoundRobinYield()

```
void  OSSchedRoundRobinYield (OS_ERR  *p_err);
```

File	Called from	Code enabled by
os_core.c	Task only	OS_CFG_SCHED_ROUND_ROBIN_EN

```
#if OS_CFG_SCHED_ROUND_ROBIN_EN > 0u
void  OSSchedRoundRobinYield (OS_ERR  *p_err)
{
    OS_RDY_LIST  *p_rdy_list;
    OS_TCB       *p_tcb;
    CPU_SR_ALLOC();
```

El código source de esta función lo encontramos en **os_core.c**

OS_ERR_NONE	if the call was successful.
OS_ERR_ROUND_ROBIN_1	if there is only one task at the current priority level that is ready-to-run.
OS_ERR_ROUND_ROBIN_DISABLED	if round-robin scheduling has not been enabled. See OSSchedRoundRobinCfg() to enable or disable.
OS_ERR_SCHED_LOCKED	if the scheduler is locked and µC/OS-III cannot switch tasks.
OS_ERR_YIELD_ISR	if OS_CFG_CALLED_FROM_ISR_CHK_EN set to 1 in os_cfg.h: if calling this function from an ISR.

p_err es el puntero a una variable que puede almacenar los códigos de error mostrados arriba. El usuario deberá chequear que el *yielding* se completó correctamente.

Conceptos e implementación

Round-Robin Scheduling. Resumen de funciones de uC/OS III

OSSchedRoundRobinCfg()

```
void  OSSchedRoundRobinCfg (CPU_BOOLEAN  en,
                            OS_TICK      dflt_time_quanta,
                            OS_ERR      *p_err)
```

File	Called from	Code enabled by
os_core.c	Task or startup code	OS_CFG_SCHED_ROUND_ROBIN_EN

```
#if OS_CFG_SCHED_ROUND_ROBIN_EN > 0u
void  OSSchedRoundRobinCfg (CPU_BOOLEAN  en,
                            OS_TICK      dflt_time_quanta,
                            OS_ERR      *p_err)
{
    CPU_SR_ALLOC();
```

El código source de esta función lo encontramos en **os_core.c**

Conceptos e implementación

Round-Robin Scheduling. Resumen de funciones de uC/OS III

OSTaskTimeQuantaSet()

```
void    OSTaskTimeQuantaSet (OS_TCB   *p_tcb,
                             OS_TICK  time_quanta,
                             OS_ERR   *p_err)
```

File	Called from	Code enabled by
os_task.c	Task only	OS_CFG_SCHED_ROUND_ROBIN_EN

```
#if OS_CFG_SCHED_ROUND_ROBIN_EN > 0u
void  OSTaskTimeQuantaSet (OS_TCB    *p_tcb,
                           OS_TICK    time_quanta,
                           OS_ERR    *p_err)
{
    CPU_SR_ALLOC();
```

El código source de esta función lo encontramos en **os_task.c**

El time quanta especificado se mide en ticks del kernel. Un valor nulo implica que se usará el valor por defecto al hacer la llamada de la función OSSchedRoundRobinCfg()

Conceptos e implementación

Round-Robin Scheduling. Implementación en Hardware

```
void InitTaskStart (void)
{
    OS_ERR os_err;

    #if OS_CFG_STAT_TASK_EN == DEF_ENABLED
        OSStatTaskCPUUsageInit(&os_err);
        OSStatReset(&os_err);

    #endif

    OSSchedRoundRobinCfg(DEF_DISABLED, (OS_TICK)10u, &os_err);
```

Comenzaremos con el scheduling de Round Robin desactivado. Previamente, para incluir el código necesario para las siguientes pruebas, será necesario habilitar el flag requerido, situado en `os_cfg.h`

`OS_CFG_SCHED_ROUND_ROBIN_EN`

```
#define OS_CFG_SCHED_ROUND_ROBIN_EN      1u    /* Include code for Round-Robin
```

Conceptos e implementación

Round-Robin Scheduling. Implementación en Hardware

```c
// Create other Tasks...
OSTaskCreate((OS_TCB      *)&task_led1_TCB,
             (CPU_CHAR    *)"Tarea LED1",
             (OS_TASK_PTR  )TASK_LED1,
             (void        *)0u,
             (OS_PRIO      )3u,
             (CPU_STK     *)&task_led1_STK[0],
             (CPU_STK_SIZE )0u,
             (CPU_STK_SIZE )1024u,
             (OS_MSG_QTY   )0u,
             (OS_TICK      )10u,
             (void        *)0,
             (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
             (OS_ERR      *)&os_err);

OSTaskCreate((OS_TCB      *)&task_led2_TCB,
             (CPU_CHAR    *)"Tarea LED2",
             (OS_TASK_PTR  )TASK_LED2,
             (void        *)0u,
             (OS_PRIO      )3u,
             (CPU_STK     *)&task_led2_STK[0],
             (CPU_STK_SIZE )0u,
             (CPU_STK_SIZE )1024u,
             (OS_MSG_QTY   )0u,
             (OS_TICK      )10u,
             (void        *)0,
             (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
             (OS_ERR      *)&os_err);
```

```c
void TASK_LED1(void)
{
    OS_ERR os_err;

    while(1)
    {
        sprintf(txdata,"Entrando en Task1. Round-Robin deshabilitado\r\n");
        EnviarString(txdata, Uart3);
        LED_RED = !LED_RED_Read;
        OSTimeDly(100, OS_OPT_TIME_DLY, &os_err);
    }
}


void TASK_LED2(void)
{
    OS_ERR os_err;

    while(1)
    {
        sprintf(txdata,"Entrando en Task2. Round-Robin deshabilitado\r\n");
        EnviarString(txdata, Uart3);
        LED_ORANGE = !LED_ORANGE_Read;
        OSTimeDly(100, OS_OPT_TIME_DLY, &os_err);
    }
}
```

Proceso de creación de las Tasks y código asignado a cada hilo.

Conceptos e implementación

Round-Robin Scheduling. Implementación en Hardware

```c
void InitTaskStart (void)
{
    OS_ERR os_err;

    #if OS_CFG_STAT_TASK_EN == DEF_ENABLED
        OSStatTaskCPUUsageInit(&os_err);
        OSStatReset(&os_err);

    #endif

    OSSchedRoundRobinCfg(DEF_DISABLED, (OS_TICK)10u, &os_err);
```

- El scheduling de Round-Robin se encuentra deshabilitado por defecto. Configurar un valor '0u' en el parámetro de entrada de la creación de cada Task (referido al Time Quanta per Task), indica al RTOS que el time slot que vamos a configurar es el valor por defecto. Sin embargo, esta acción no inicializa Round-Robin.

- En la anterior diapositiva hemos creado dos Tasks (destinadas al control del LED1 y LED2). Como parámetros de inicialización, mantenemos ambas con la misma prioridad (3). Ploteando los valores de la Uart por el puerto COM, veremos el siguiente comportamiento. Recordad, que en este punto Round Robin aún sigue deshabilitado.

```
***** EVALUATION BOARD v1. TEST RTOS AND 6 LEDS *****

Entrando en Task1. Round-Robin deshabilitado
Entrando en Task2. Round-Robin deshabilitado
Entrando en Task1. Round-Robin deshabilitado
Entrando en Task2. Round-Robin deshabilitado
Entrando en Task1. Round-Robin deshabilitado
Entrando en Task2. Round-Robin deshabilitado
Entrando en Task1. Round-Robin deshabilitado
```

- Como es de esperar, el primer mensaje que vemos por pantalla corresponde a Task1. Ambas Tasks comparten el mismo nivel de prioridad. Sin embargo, Task1 es la primera en asignarse a la lista de Tareas creadas, además de ser la primera a la que se le asigna su TCB.

Conceptos e implementación

Round-Robin Scheduling. Implementación en Hardware

```
// Create other Tasks...
OSTaskCreate((OS_TCB      *)&task_led1_TCB,
             (CPU_CHAR    *)"Tarea LED1",
             (OS_TASK_PTR  )TASK_LED1,
             (void        *)0u,
             (OS_PRIO      )4u,
             (CPU_STK     *)&task_led1_STK[0],
             (CPU_STK_SIZE )0u,
             (CPU_STK_SIZE )1024u,
             (OS_MSG_QTY   )0u,
             (OS_TICK      )10u,
             (void        *)0,
             (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
             (OS_ERR      *)&os_err);

OSTaskCreate((OS_TCB      *)&task_led2_TCB,
             (CPU_CHAR    *)"Tarea LED2",
             (OS_TASK_PTR  )TASK_LED2,
             (void        *)0u,
             (OS_PRIO      )3u,
             (CPU_STK     *)&task_led2_STK[0],
             (CPU_STK_SIZE )0u,
             (CPU_STK_SIZE )1024u,
             (OS_MSG_QTY   )0u,
             (OS_TICK      )10u,
             (void        *)0,
             (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
             (OS_ERR      *)&os_err);
```

Menor prioridad (Número más alto)

- Cambiando el valor de las prioridades (Task 2 con mayor prioridad que Task1), vemos el siguiente resultado:

```
***** EVALUATION BOARD v1. TEST RTOS AND 6 LEDS *****

Entrando en Task2. Round-Robin deshabilitado
Entrando en Task1. Round-Robin deshabilitado
Entrando en Task2. Round-Robin deshabilitado
Entrando en Task1. Round-Robin deshabilitado
Entrando en Task2. Round-Robin deshabilitado
Entrando en Task1. Round-Robin deshabilitado
```

- Vemos que Task2 (a pesar de ser creada después de Task1), es la primera en hacer uso del puerto Uart, dado su nivel de prioridad respecto a Task1.

Conceptos e implementación

Round-Robin Scheduling. Implementación en Hardware

- Ahora iniciaremos el RTOS con scheduling Round-Robin. Recordad, que Round-Robin puede ejecutarse siempre y cuando existan, al menos, dos Tasks con idéntica prioridad. Como ejemplo, dejaremos ambas Tasks al nivel 3. Realizaremos las siguientes modificaciones de código:

```
OSSchedRoundRobinCfg(DEF_ENABLED, (OS_TICK)10u, &os_err);

// Create other Tasks...
OSTaskCreate((OS_TCB      *)&task_led1_TCB,
             (CPU_CHAR    *)"Tarea LED1",
             (OS_TASK_PTR  )TASK_LED1,
             (void        *)0u,
             (OS_PRIO      )3u,
             (CPU_STK     *)&task_led1_STK[0],
             (CPU_STK_SIZE )0u,
             (CPU_STK_SIZE )1024u,
             (OS_MSG_QTY   )0u,
             (OS_TICK      )10u,
             (void        *)0,
             (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
             (OS_ERR      *)&os_err);

OSTaskCreate((OS_TCB      *)&task_led2_TCB,
             (CPU_CHAR    *)"Tarea LED2",
             (OS_TASK_PTR  )TASK_LED2,
             (void        *)0u,
             (OS_PRIO      )3u,
             (CPU_STK     *)&task_led2_STK[0],
             (CPU_STK_SIZE )0u,
             (CPU_STK_SIZE )1024u,
             (OS_MSG_QTY   )0u,
             (OS_TICK      )10u,
             (void        *)0,
             (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
             (OS_ERR      *)&os_err);
```

Para iniciar Round-Robin, utilizaremos la función de configuración que hemos visto anteriormente. Esta función presenta dos parámetros de entrada y uno de salida. DEF_ENABLED simplemente habilita este scheduling. (OS_TICK)10u indica al planificador de tareas que deseamos un Time Quanta general de 10 ticks (10ms para un Tick de 1ms) para cada Tarea.

¿Qué ocurrirá con ambos mensajes con Round-Robin habilitado y 921600 bits/segundo de tasa de datos?

```
***** EVALUATION BOARD v1. TEST RTOS AND 6 LEDS *****

Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
_
```

:00:02 conectado | Autodetect. | 921600 8-N-1 | DESPLAZAR | MAY | NUM | Capturar

- La transmisión de datos y el time delay eficiente son lo suficientemente elevados como para permitir al sistema realizar las transmisiones, sin solaparse una con la otra.

Conceptos e implementación

Round-Robin Scheduling. Implementación en Hardware

¿Qué ocurrirá con ambos mensajes con Round-Robin habilitado y 921600 bits/segundo de tasa de datos, pero esta vez manteniendo un Time Quanta de 1u?

```
***** EVALUATION BOARD v1. TEST RTOS AND 6 LEDS *****

Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
_
```

0:00:02 conectado | Autodetect. | 921600 8-N-1 | DESPLAZAR | MAY | NUM | Capturar

- Un Time Quanta de 1u, implica un slot de tiempo de 1 ms antes de que se produzca un *Context Switch* entre ambas Tasks. Aproximadamente, la cadena de mensaje consta de 48 caracteres. Esto implica que, a una velocidad de 921600, el tiempo total de envío de todos los caracteres, incluyendo \r\n, será de:

$$\text{Tiempo de envío} = \frac{10\frac{bits}{carac}}{921600\frac{bits}{seg}} * 48\ carac \approx 520{,}83\ useg$$

- Como vemos, la transmisión se realiza, aproximadamente, 2 veces más rápido que la expiración del Time Quanta. Por tanto, los mensajes aún no se solapan entre ellos. (Para un código sencillo, es fácil determinar este comportamiento. En casos más complejos, hará falta un visualizador de traza).

Conceptos e implementación

Round-Robin Scheduling. Implementación en Hardware

¿Qué ocurrirá con ambos mensajes con Round-Robin habilitado y 9600 bits/segundo de tasa de datos, manteniendo un Time Quanta de 10u?

```
***** EVALUATION BOARD v1. TEST RTOS AND 6 LEDS *****

Entrando en Task1.Entrando e Round-Ron Task2. Rbin habilound-Robinitado
 habilitado
in habilitn Entrando eTask2. Roun Task2. nd-Robin hRound-Robabilitado
          ado
Entrando en Task1.Entrando e Round-Ron Task2. Rbin habiliound-Robitado
n habilitado
Entrando en TEntrando eask2. Rounn Task2. d-Robin haRound-Robbilitado
in habilitado
Entrando Entrando een Task2. n Task2. Round-RobiRound-Robin habilitn habilitaado

do
Entrando en TaEntrando esk2. Rounn Task2. Rd-Robin haound-Robibilitado
n habilitado
Entrando Entrando een Task2. n Task2. Round-RobiRound-Robin habilitn habilitaado

do
Entrando en TaEntrando esk2. Rounn Tas_
```

:00:22 conectado | Autodetect. | 9600 8-N-1 | DESPLAZAR | MAY | NUM | Capturar | Imprimir

- La transmisión de datos se queda corrupta, pues ambas Tasks hacen uso del recurso compartido (puerto Uart) en instantes donde hay una comunicación aún en proceso. Tened en cuenta además que usamos el mismo buffer de datos txdata[] para cargar ambos mensajes.

- Este efecto lo vemos al haber cambiado a una velocidad de datos bastante más baja (9600 baudios), a lo que se añade los tiempos de Time Quanta y delays configurados. Esto genera el ambiente propicio para que se observe este efecto.

- Esto no debe ser un problema cuando trabajamos con RTOS. Más adelante veremos los objetos de Kernel encargados del control de recursos compartidos (Mutex, semáforos binarios, queues, etc..)

Conceptos e implementación

Round-Robin Scheduling. Implementación en Hardware

¿Qué ocurrirá con ambos mensajes con Round-Robin habilitado y 9600 bits/segundo de tasa de datos, manteniendo un Time Quanta de `100u` y `OS_Time_Delay de 1000u`?

```
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habiliEntrando en Task2. Round-Robin habilitado
tado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-RobinEntrando en Task1. Round-Robin habilitado
 habilitado
Entrando en Task1. Round-Robin habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-RobiEntrando en Task2. Round-Robin habilitado
n habilitado
Entrando en Task2. Round-Robin habilitado
Entrando en Task1. Round-Robin habilitado
```

Desconectado | Autodetect. | 9600 8-N-1 | DESPLAZAR | MAY | NUM | Capturar | Imprimir

- Como vemos, el efecto no es tan pronunciado como en el caso anterior. Sin embargo, dados los timings existen puntos en los que aún se solapan ambas transmisiones.

Conceptos e implementación

Objetos de Kernel y features en cada versión del RTOS.

Feature	µC/OS	µC/OS-II	>µC/OS-III
Year introduced	1992	1998	2009
Book	Yes	Yes	Yes
Source code available	Yes	Yes	Yes
Preemptive Multitasking	Yes	Yes	Yes
Maximum number of tasks	64	255	Unlimited
Number of tasks at each priority level	1	1	Unlimited
Round Robin Scheduling	No	No	Yes
Semaphores	Yes	Yes	Yes
Mutual Exclusion Semaphores	No	Yes	Yes (Nestable)
Event Flags	No	Yes	Yes
Message Mailboxes	Yes	Yes	No (not needed)
Message Queues	Yes	Yes	Yes
Fixed Sized Memory Management	No	Yes	Yes
Signal a task without requiring a semaphore	No	No	Yes
Option to Post/Signal without scheduling	No	No	Yes

Conceptos e implementación

Internal Tasks (Tasks no pertenecientes a la propia aplicación)

Internal Tasks

During initialization, µC/OS-III creates between 0 and 4 internal tasks depending on configuration constants found in os_cfg.h:

- `OS_CFG_TASK_IDLE_EN` enables `OS_IdleTask()`
- `OS_CFG_STAT_TASK_EN` enables `OS_StatTask()`
- `OS_CFG_TASK_TICK_EN` enables `OS_TickTask()`
- `OS_CFG_TMR_EN` enables `OS_TmrTask()`

Conceptos e implementación

Idle Task

```
void  OS_IdleTask (void *p_arg)
          {
              while (DEF_ON) {
                  CPU_CRITICAL_ENTER();
                  OSIdleTaskCtr++;
                  OSStatTaskCtr++;
                  CPU_CRITICAL_EXIT();
                  OSIdleTaskHook();
              }
          }
```

Prioridad de la Idle task -> `OS_CFG_PRIO_MAX-1`

Disponible en -> `os_core.c`

Configuración -> `os_cfg.h`

Conceptos e implementación

Statistic Task

```
OS_ERR os_err;


// Create Statistic Task Initialization
OSTaskCreate(
                (OS_TCB      *)&InitTaskStart_TCB,
                (CPU_CHAR    *)"Statistic Task",
                (OS_TASK_PTR  )InitTaskStart,
                (void        *)0,
                (OS_PRIO      )5u,
                (CPU_STK     *)&InitTaskStart_STK[0],
                (CPU_STK_SIZE )0u,
                (CPU_STK_SIZE )10000u,
                (OS_MSG_QTY   )0u,
                (OS_TICK      )0u,
                (void        *)0,
                (OS_OPT       )(OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR),
                (OS_ERR      *)&os_err
            );
```

Prioridad de la static task -> `OS_CFG_STAT_TASK_PRIO`

Habilitación -> `OS_CFG_STAT_TASK_EN`

Configuración -> `os_cfg.h`

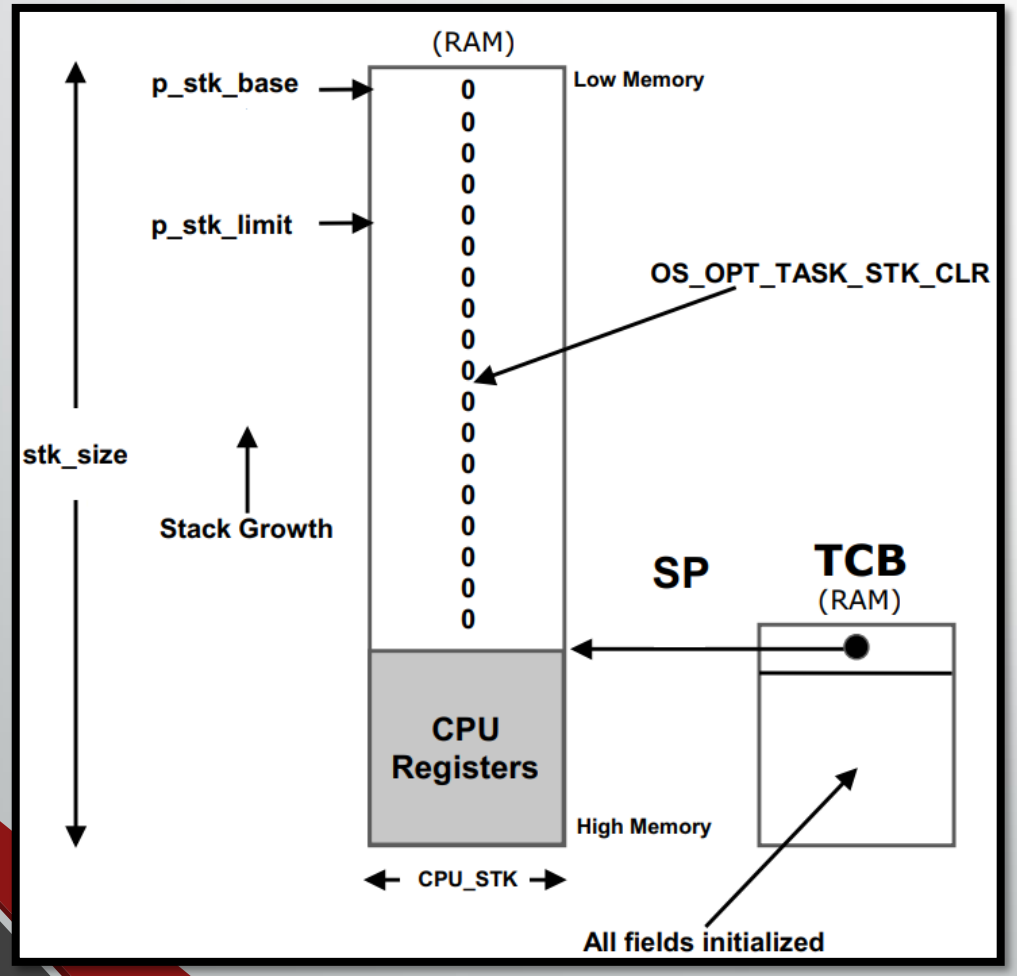
`os_cfg_app.h`

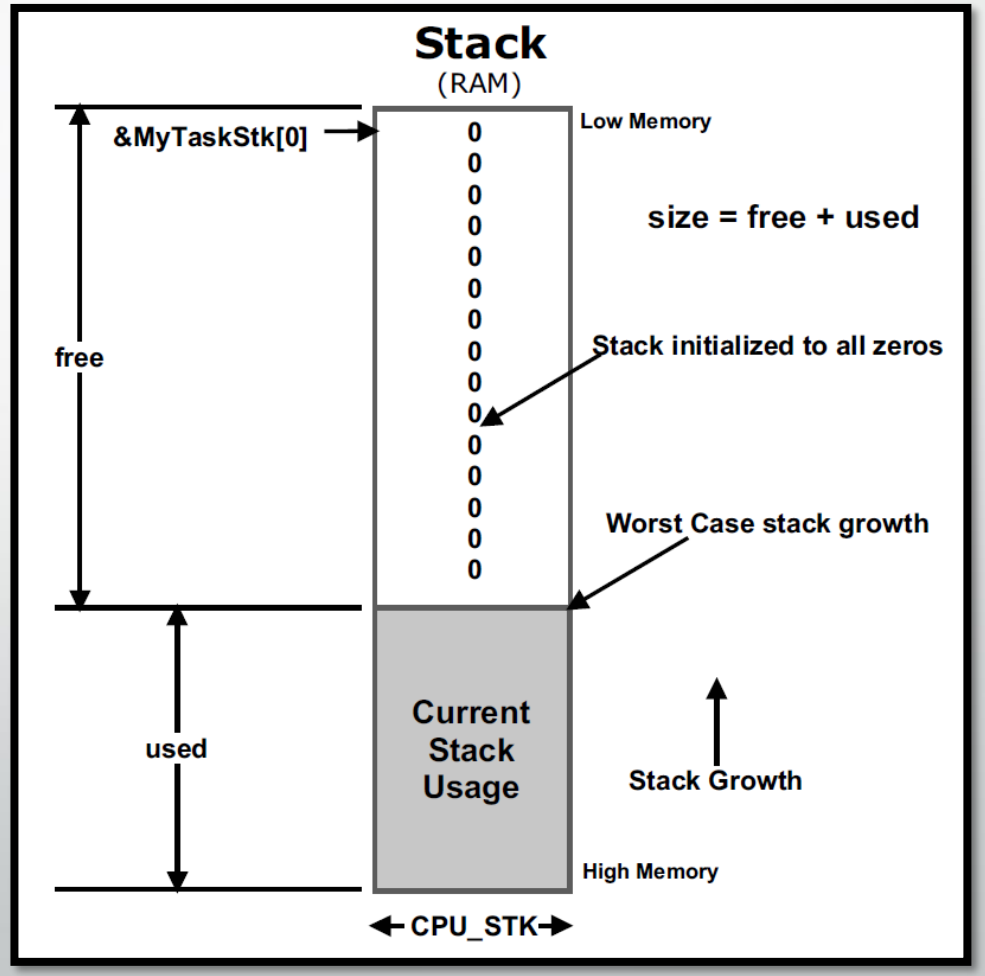
Necesitamos llamar previamente a la función -> `OSStatTaskCPUUsageInit()`

$$\text{OSStatTaskCPU Usage}_{(\%)} = \left(100 - \frac{100 \times \text{OSStatTaskCtr}}{\text{OSStatTaskCtrMax}}\right)$$

Conceptos e implementación

Task Stack. Variables accesibles a través del TCB de la propia tarea.





Conceptos e implementación

Scheduling: Ready List

Una vez hemos creado las Tasks de nuestra aplicación (*OSTaskCreate()*), el kernel gestiona las tareas que están listas para ejecutarse (*Ready to Run* State) mediante la ***Ready List.***

La Ready List consta de dos partes diferenciadas:

- **Bitmap (Prio List)** con los niveles de prioridad, correspondientes a las Tasks listas para ser ejecutadas.
- **Tabla (Ready List)** que contiene los punteros o direccionamientos a cada una de las Tasks anteriores (ready).

Recordemos, que el número máximo de niveles de prioridad es establecido mediante ***OS_CFG_PRIO_MAX***, dentro del header de configuración del RTOS ***os_cfg.h***

En uC/OS-III, valores numéricos crecientes indican niveles de prioridad más bajos. Por tanto, un nivel numérico 0 implica la prioridad más elevada del sistema. El valor numérico más alto, funcional, es OS_CFG_PRIO_MAX – 1 (es decir, la prioridad más baja es asignada únicamente a Task Idle)

Conceptos e implementación

Scheduling: Ready List

Antes de continuar, recordemos que el tamaño de palabra asignado al RTOS se realiza con la configuración de **CPU_CFG_DATA_SIZE**, en el header **cpu.h**. En el caso del PIC32, por defecto se asigna 32-bit.

```
*********************************************************************************************************
*                                          CPU WORD CONFIGURATION
*
* Note(s) : (1) Configure CPU_CFG_ADDR_SIZE & CPU_CFG_DATA_SIZE with CPU's word sizes :
*
*                   CPU_WORD_SIZE_08             8-bit word size
*                   CPU_WORD_SIZE_16            16-bit word size
*                   CPU_WORD_SIZE_32            32-bit word size
*                   CPU_WORD_SIZE_64            64-bit word size            See Note #1a
*
*               (a) 64-bit word size NOT currently supported.
*
*           (2) Configure CPU_CFG_ENDIAN_TYPE with CPU's data-word-memory order :
*
*               (a) CPU_ENDIAN_TYPE_BIG         Big-   endian word order (CPU words' most  significant
*                                                                         octet @ lowest memory address)
*               (b) CPU_ENDIAN_TYPE_LITTLE      Little-endian word order (CPU words' least significant
*                                                                         octet @ lowest memory address)
*********************************************************************************************************
*/

                                                        /* Define  CPU         word sizes (see Note #1) :       */
#define  CPU_CFG_ADDR_SIZE              CPU_WORD_SIZE_32   /* Defines CPU address word size  (in octets).          */
#define  CPU_CFG_DATA_SIZE              CPU_WORD_SIZE_32   /* Defines CPU data    word size  (in octets).          */
```

Conceptos e implementación

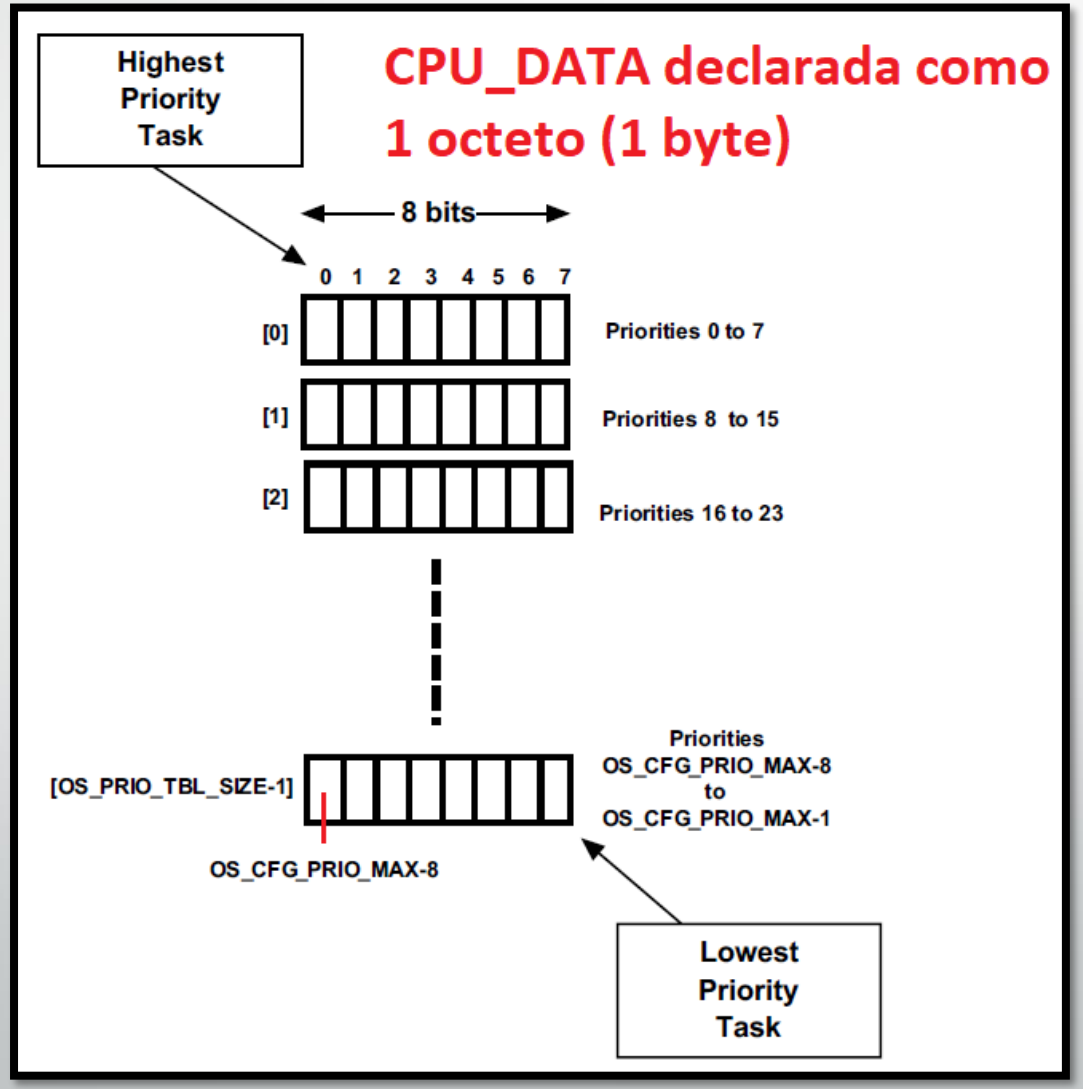
Scheduling: Ready List

Por otro lado, la definición de los tamaños de palabra disponibles (y que puedan ser posibles) se realizan en el header **cpu_def.h**

```
#define  CPU_WORD_SIZE_08                    1u   /* ---------------------- CPU WORD SIZE ------------------------ */
#define  CPU_WORD_SIZE_16                    2u   /*  8-bit word size (in octets).                                  */
#define  CPU_WORD_SIZE_32                    4u   /* 16-bit word size (in octets).                                  */
#define  CPU_WORD_SIZE_64                    8u   /* 32-bit word size (in octets).                                  */
                                                  /* 64-bit word size (in octets).                                  */
```

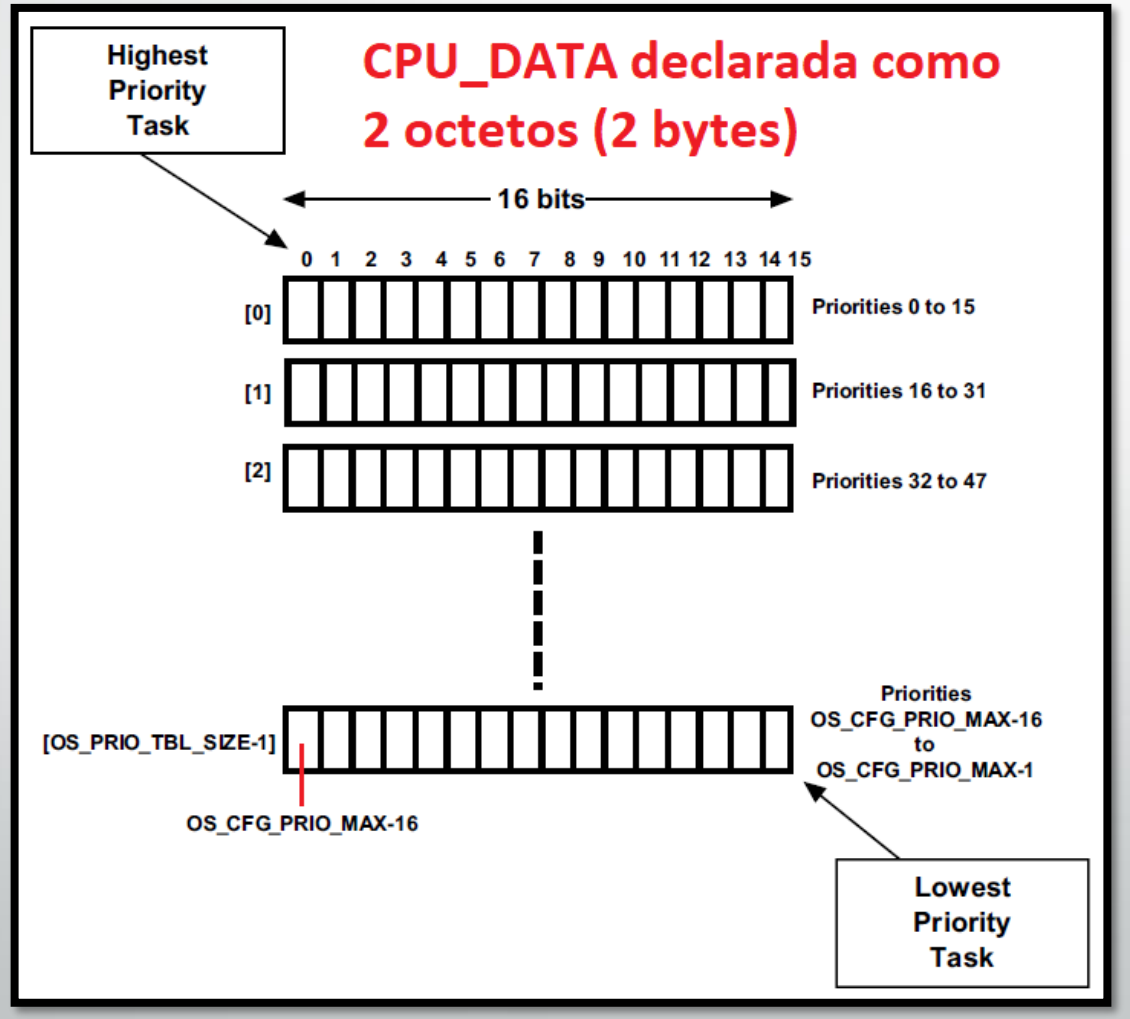
Conceptos e implementación

Scheduling: Ready List



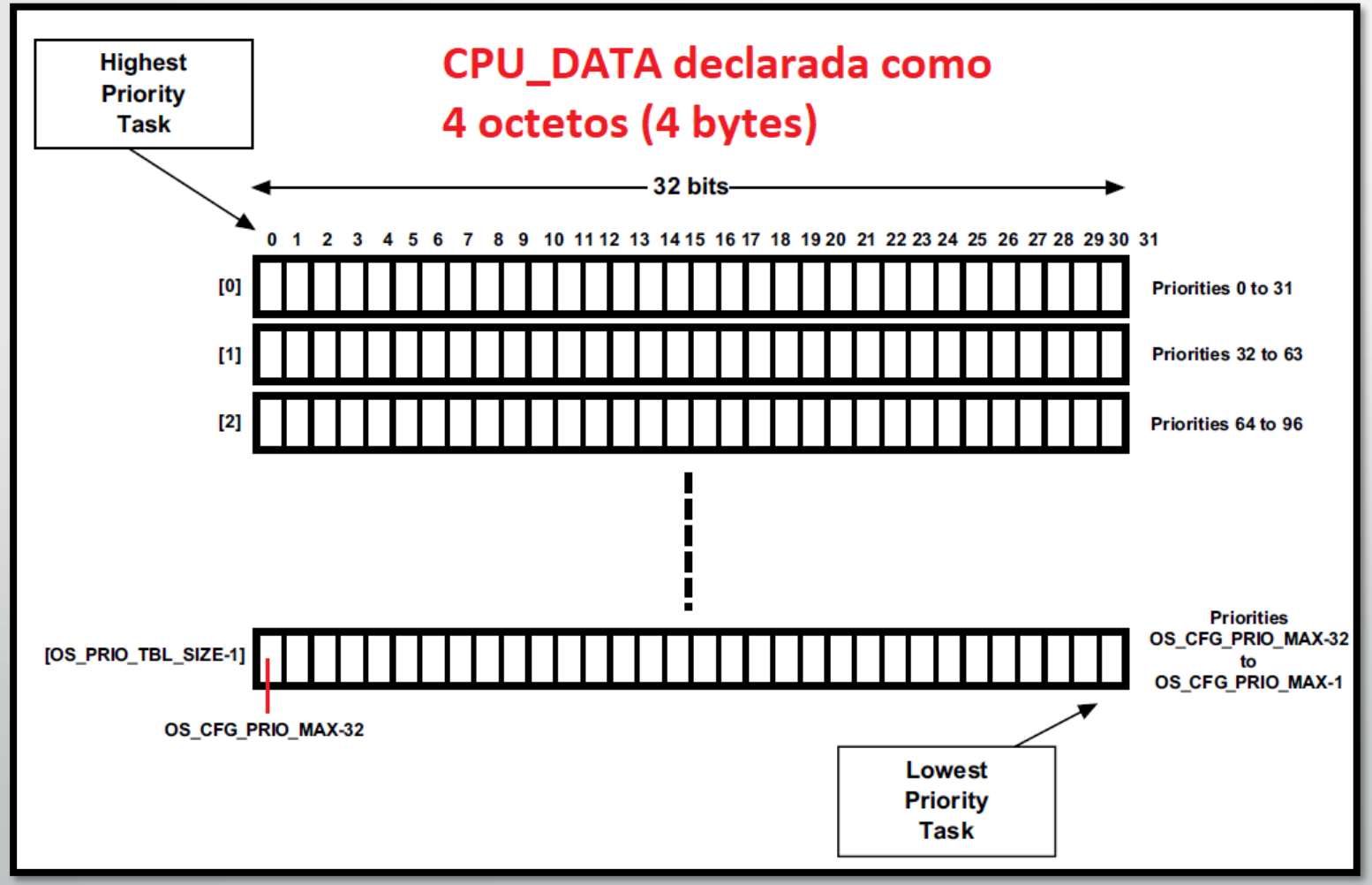
Conceptos e implementación

Scheduling: Ready List



Conceptos e implementación

Scheduling: Ready List



Conceptos e implementación

Scheduling: Ready List

Para realizar un escaneado de la prioridad más alta, enlazada con Tasks *Ready to Run*, se dispone de una API propia del RTOS encargada de hacer un barrido a través de las tablas anteriores, en busca del primer slot con un bit set (1).

OS_PrioGetHighest() se encarga de este objetivo. Tambíen se disponen de otras dos funciones encargadas de manejar la configuración de bits de cada prioridad, de manera flexible. Estas funciones podemos encontrarlas en el archivo source **OS_PRIO.C**

Function	Description
`OS_PrioGetHighest()`	Find the highest priority level
`OS_PrioInsert()`	Set bit corresponding to priority level in the bitmap table
`OS_PrioRemove()`	Clear bit corresponding to priority level in the bitmap table

Conceptos e implementación

Scheduling: Ready List

La función **OS_PrioGetHighest()** presenta el siguiente código fuente. Lo primero se define la propia tabla donde se lleva el registro bitmap de prioridades. El tamaño a asignar (OS_PRIO_TBL_SIZE) depende del valor numérico introducido para indicar el número total de prioridades y del tamaño de Word (CPU_DATA) utilizado por el RTOS.

```
CPU_DATA    OSPrioTbl[OS_PRIO_TBL_SIZE];
```

```
#define  OS_PRIO_TBL_SIZE          ((OS_CFG_PRIO_MAX - 1u) / (sizeof(CPU_DATA) * 8u) + 1u)
```

Definición en ***os.h***

```
OS_PRIO  OS_PrioGetHighest (void)
{
    CPU_DATA  *p_tbl;
    OS_PRIO    prio;



    prio  = (OS_PRIO)0;
    p_tbl = &OSPrioTbl[0];
    while (*p_tbl == (CPU_DATA)0) {
        prio += sizeof(CPU_DATA) * 8u;
        p_tbl++;
    }
    prio += (OS_PRIO)CPU_CntLeadZeros(*p_tbl);
    return (prio);
}
```

Definición en ***cpu.h***

```
#if     (CPU_CFG_DATA_SIZE == CPU_WORD_SIZE_32)
typedef  CPU_INT32U  CPU_DATA;
#elif   (CPU_CFG_DATA_SIZE == CPU_WORD_SIZE_16)
typedef  CPU_INT16U  CPU_DATA;
#else
typedef  CPU_INT08U  CPU_DATA;
#endif
```

```
typedef            void        CPU_VOID;
typedef            char        CPU_CHAR;
typedef  unsigned  char        CPU_BOOLEAN;
typedef  unsigned  char        CPU_INT08U;
typedef    signed  char        CPU_INT08S;
typedef  unsigned  short       CPU_INT16U;
typedef    signed  short       CPU_INT16S;
typedef  unsigned  int         CPU_INT32U;
typedef    signed  int         CPU_INT32S;
typedef  unsigned  long  long  CPU_INT64U;
typedef    signed  long  long  CPU_INT64S;

typedef            float       CPU_FP32;
typedef            double      CPU_FP64;


typedef  volatile  CPU_INT08U  CPU_REG08;
typedef  volatile  CPU_INT16U  CPU_REG16;
typedef  volatile  CPU_INT32U  CPU_REG32;
typedef  volatile  CPU_INT64U  CPU_REG64;
```

Conceptos e implementación

Scheduling: Ready List

```
CPU_DATA    OSPrioTbl[OS_PRIO_TBL_SIZE];
```

```
OS_PRIO  OS_PrioGetHighest (void)
{
    CPU_DATA  *p_tbl;
    OS_PRIO    prio;


    prio  = (OS_PRIO)0;
    p_tbl = &OSPrioTbl[0];
    while (*p_tbl == (CPU_DATA)0) {
        prio += sizeof(CPU_DATA) * 8u;
        p_tbl++;
    }
    prio += (OS_PRIO)CPU_CntLeadZeros(*p_tbl);
    return (prio);
}
```

- Lo primero es definir el puntero que va contener la dirección de memoria de la tabla o bitmap. Por otro lado, se crea una variable '**prio**', que contendrá la prioridad más elevada con Tasks enlazadas en *Ready to Run state.*

- Como inicialización, se almacena un valor de reset de 0 en la variable 'prio', definida anteriormente. Posteriormente, almacenamos en el puntero la dirección de memoria del inicio del bitmap de prioridades. Este puntero apunta a un tamaño de dirección correspondiente a CPU_DATA (4 bytes). El bucle *while* se utiliza para realizar un escaneado de un bit set (1), el cual nunca es infinito gracias a la *Task IDLE*, que toma el valor de prioridad más bajo y siempre tiene un valor no-nulo en el bitmap.

- Cuando se encuentra un valor **p_tbl* non-zero, se llama a la función CLZ. Esta función devuelve el valor de prioridad más elevado con Tasks listas para ser ejecutadas, mediante un conteo hasta encontrar un bit set. Este valor de conteo corresponde con el valor de prioridad más elevado con Tasks *Ready to Run*

Conceptos e implementación

Scheduling: Ready List

La función **OS_PrioGetHighest()** presenta el siguiente código fuente. Lo primero se define la propia tabla donde se lleva el registro bitmap de prioridades. El tamaño a asignar (OS_PRIO_TBL_SIZE) depende del valor numérico introducido para indicar el número total de prioridades y del tamaño de Word (CPU_DATA) utilizado por el RTOS.

EXPLICACIÓN CON EJEMPLO EN CLASE/VIDEOCONFERENCIA

```
CPU_DATA    OSPrioTbl[OS_PRIO_TBL_SIZE];
```

```
OS_PRIO  OS_PrioGetHighest (void)
{
    CPU_DATA  *p_tbl;
    OS_PRIO    prio;



    prio  = (OS_PRIO)0;
    p_tbl = &OSPrioTbl[0];
    while (*p_tbl == (CPU_DATA)0) {
        prio += sizeof(CPU_DATA) * 8u;
        p_tbl++;
    }
    prio += (OS_PRIO)CPU_CntLeadZeros(*p_tbl);
    return (prio);
}
```

Conceptos e implementación

Scheduling: Ready List

Una vez se realiza el chequeo del bitmap para determinar la prioridad más elevada con Tasks en estado *Ready to Run* (aún no hemos gestionando las Tasks que se encuentran ligadas a esta prioridad), se procede a actualizar la Ready List.

Anteriormente hemos visto la **OSPrioTbl[]**, que contiene el bitmap de prioridades. Por otra parte, uC/OS-III cuenta con la lista o array **OSRdyTbl[],** la cual forma la 2ª parte del proceso.

OSRdyTbl[] se define como un array que contiene *OS_CFG_PRIO_MAX* elementos o entradas. Este array es definido con el data type OS_RDY_LIST, en el header *OS.h*

OS_RDY_LIST es, realmente, una estructura de datos (como TCBs) que contiene los siguientes 3 campos:

- .Entries -> Contiene el número de Tasks *Ready-to-Run* para un nivel de prioridad determinado (correspondiente al elemento de la lista al que nos estemos refiriendo).
- .TailPtr -> Puntero con la dirección a la Task *Ready-to-Run* a la cola de Tasks a ejecutar, para un nivel de prioridad determinando. En realidad, apunta al TCB de una Task.
- .HeadPtr -> Puntero con la dirección a la Task *Ready-to-Run* a la cabeza de Tareas a ejecutar, para un nivel de prioridad determinado. En realidad, apunta al TCB de una Task.

Conceptos e implementación

Scheduling: Ready List

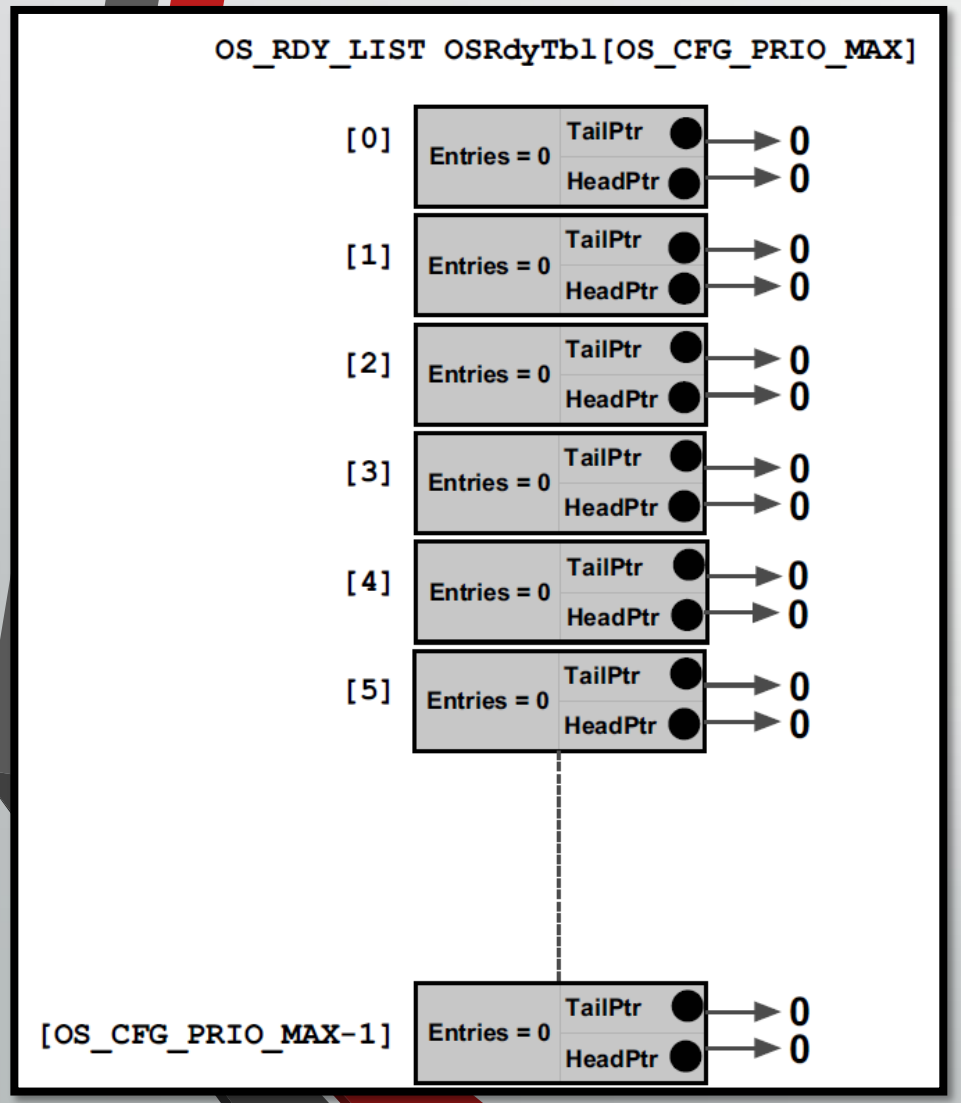
Como mencionábamos anteriormente, OSRdyTbl[] es definida como una estructura de datos que contiene los 3 elementos que hemos visto. El index o elemento del array indica el nivel de prioridad al que nos estamos refiriendo, de modo que, a modo de ejemplo:

Si creamos una Task con nivel de prioridad 3, entonces esta Task será insertada en la tabla, concretamente en OSRdyTbl[3], siempre y cuando esta Task este lista para ser ejecutada (*Ready-to-Run*). uC/OS-III dispone de varias funciones para controlar la inicialización de la tabla y sus elementos, insertar nuevas Tasks añadidas al proceso, eliminarlas, moverlas a la Head o Tail y viceversa. Estas funciones son **internas** y usadas por el propio RTOS, de modo que **no** deben llamarse por las Tasks de nuestra aplicación.

Function	Description
`OS_RdyListInit()`	Initialize the ready list to "empty" (see Figure 6-4)
`OS_RdyListInsert()`	Insert a TCB into the ready list
`OS_RdyListInsertHead()`	Insert a TCB at the head of the list
`OS_RdyListInsertTail()`	Insert a TCB at the tail of the list
`OS_RdyListMoveHeadToTail()`	Move a TCB from the head to the tail of the list
`OS_RdyListRemove()`	Remove a TCB from the ready list

Conceptos e implementación

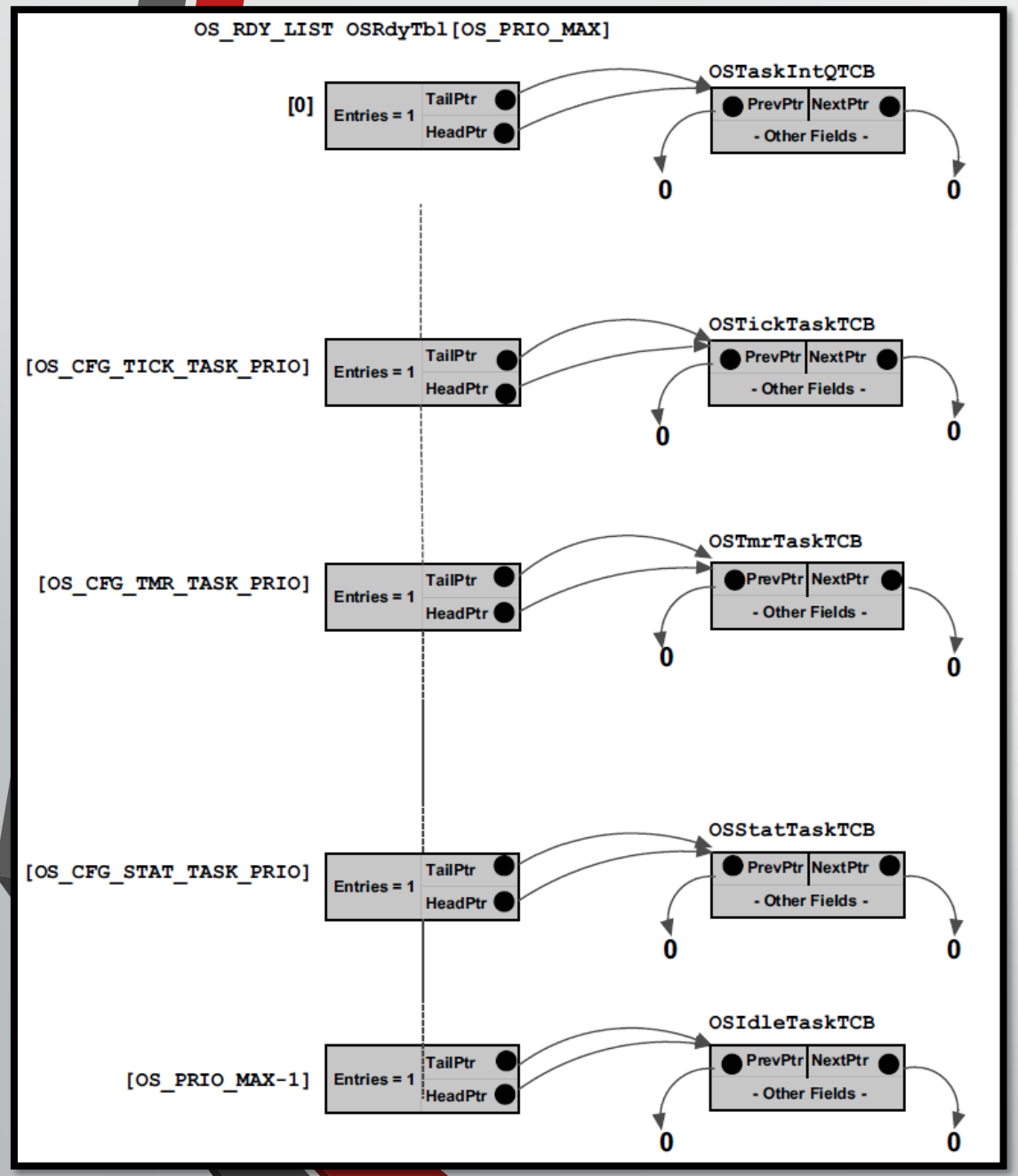
Scheduling: Ready List



En la siguiente figura se muestra un ejemplo de la lista de Tasks *Ready-to-Run* justo en la inicialización del RTOS, al ejecutar OSInit(). Justo en la inicialización, el número de entries en este estado, para cada prioridad, es cero (empty), ya que aún no se ha ejecutado OSStart(), y por tanto no existe ninguna Task lista para ser ejecutada, a ningún nivel de prioridad.

Puesto que no existen un número de entries > 1, los punteros de Head y Tail apuntan a NULL (0). Esto indica que no se esta apuntando al TCB de ninguna Task en particular (ni a la cabeza ni a la cola de la lista)

Conceptos e implementación



Scheduling: Ready List

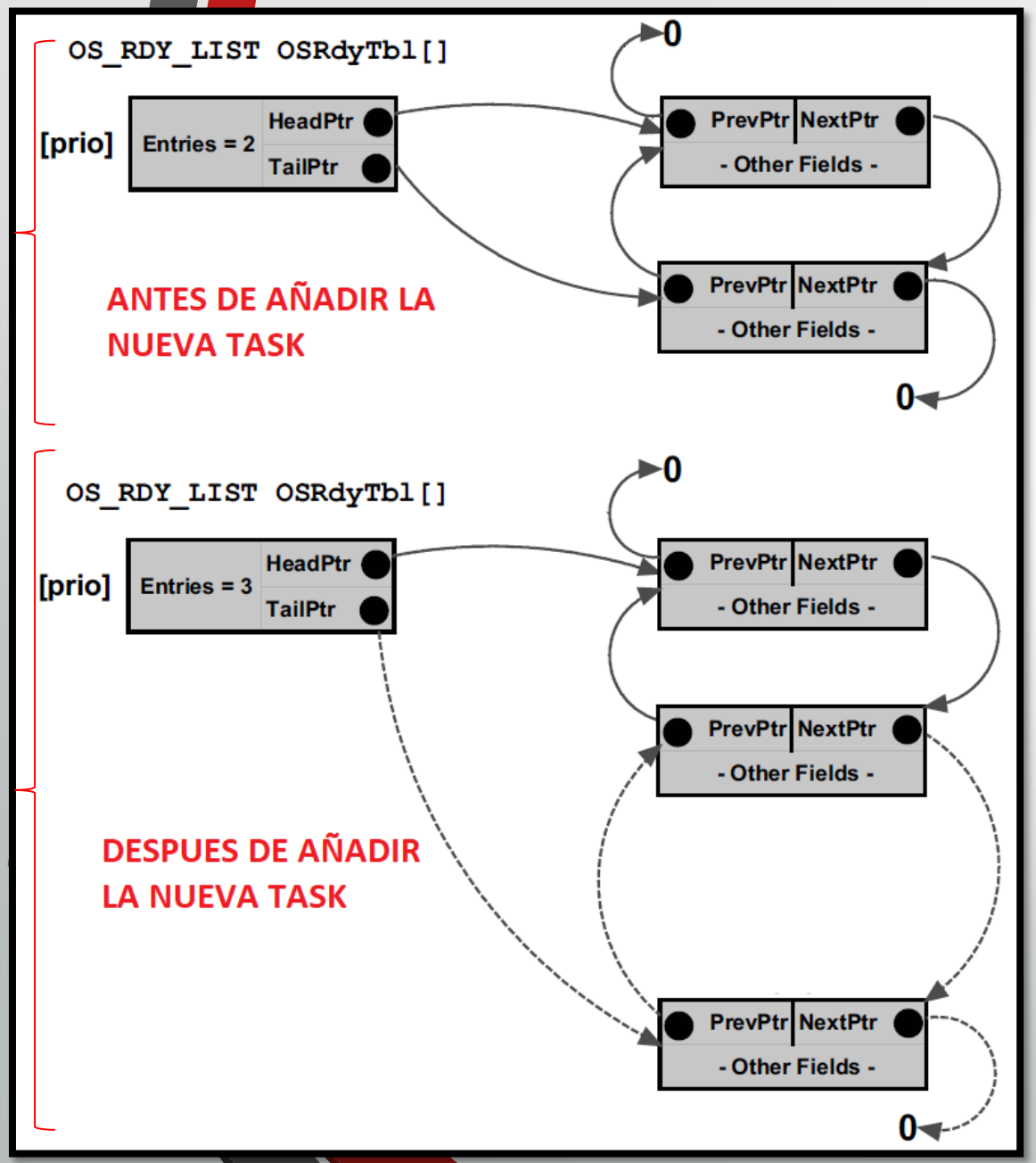
- En la siguiente figura se muestra un ejemplo de la lista de Tasks *Ready-to-Run*, después de ejecutar `OSStart()`. Se asume que se han habilitado, para este ejemplo, todas las Tasks internas (ninguna Task de APP). Para este ejemplo, se ha configurado las Tasks internas con niveles de prioridad diferentes y únicos.

- Dentro de cada TCB, disponemos de dos punteros, que se utilizan para enlazar y direccionar Tasks del mismo nivel de prioridad de la lista ready to run:
  - .PrevPtr
  - .NextPtr

- Para la IDLE Task, los punteros siempre direccionan a NULL, ya que no puede existir ninguna otra Task asociada a este nivel de prioridad (OS_CFG_PRIO_MAX - 1), y por tanto el número de entries siempre será igual a 1 en OSRdyTbl[OS_CFG_PRIO_MAX - 1].

Conceptos e implementación

Scheduling: Ready List



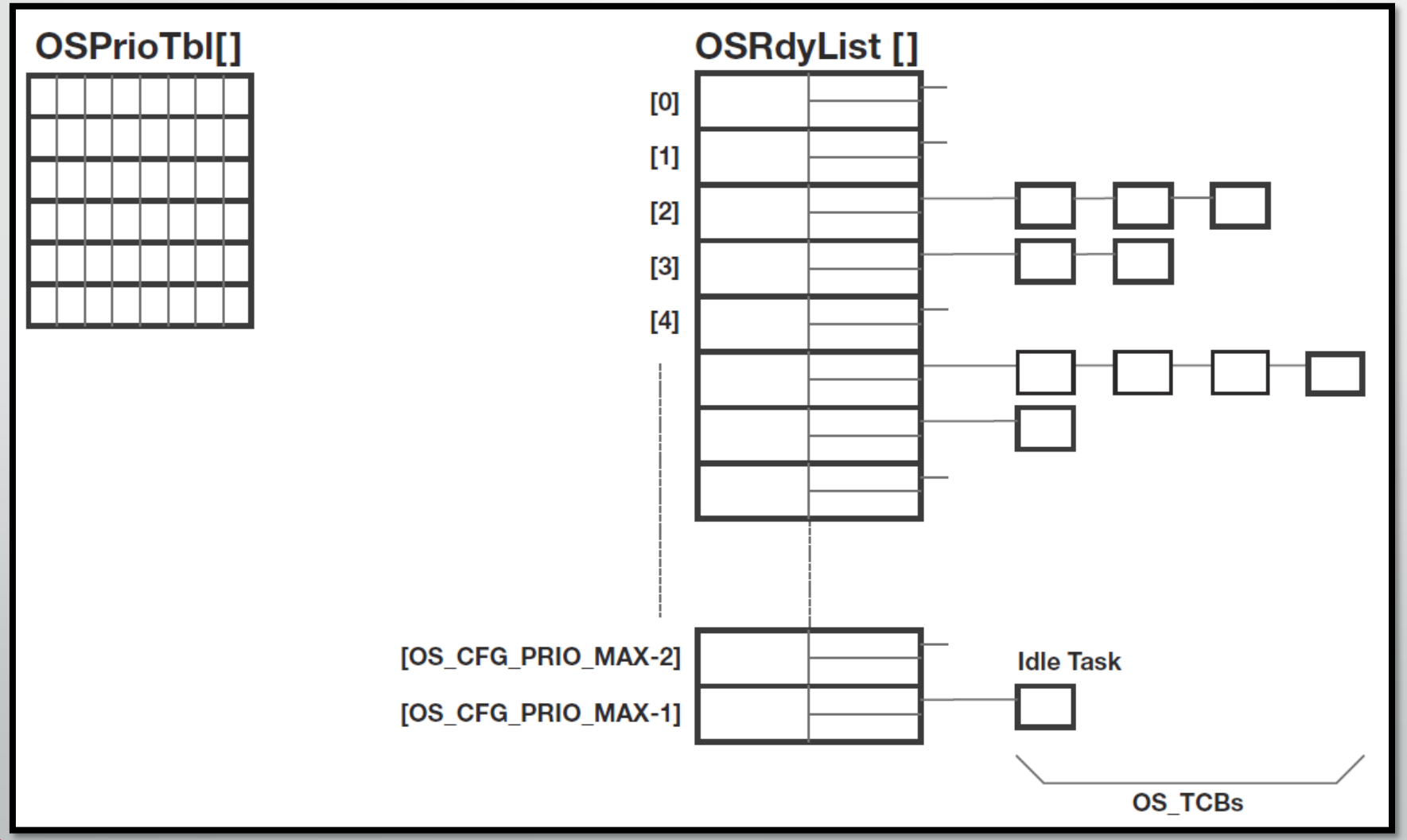
- En la siguiente figura se muestran los cambios de añadir una Task (de APP, por ejemplo) a la ready list (ya inicializada). Se muestra "el antes y el después" del proceso.

- Antes del proceso, disponemos de 2 Tasks listas para ser ejecutadas. Como vemos, los punteros Head y Tail de la Ready List, para el nivel de prioridad 'prio', apuntan a los TCBs de ambas Tasks, respectivamente. Puesto que partimos del mismo nivel de prioridad para ambas, el orden en el que se establece esta lista es dependiente del orden en el que se crearon las Tasks, en las circunstancias más simples.

- Después de añadir una nueva Task, el número de *entries* asciende a 3. El apuntamiento de los punteros forma la estructura que vemos en la figura, donde se lleva un tracking (del TCB anterior y siguiente) de la ready list, de este nivel de prioridad en concreto.

Conceptos e implementación

Scheduling: Priority bitmap and Ready List



Conceptos e implementación

Scheduling: Internals

Las funciones principales, encargadas de realizar el *scheduling*, son las siguientes APIs del RTOS:

- **OSSched()** -> Llamada a nivel de Tasks.
- **OSIntExit()** -> Llamada a nivel de ISRs.

El código de ambas funciones lo podemos encontrar en **os_core.c**

Conceptos e implementación

Scheduling: Internals

- **OSSched()** -> Llamada a nivel de Tasks.

OSSched() es una API propia del RTOS, encargada de comprobar y ejecutar una nueva planificación o scheduling. Esta función es llamada a nivel de Tasks a través de los múltiples servicios de los que dispone uC/OS-III, en los llamados *Scheduling Points* (que serán detallados más adelante).

OSSched() es, por tanto, la función principal de scheduling llamada a nivel de Tasks para determinar si una nueva Task de mayor prioridad ha cambiado al estado *Ready-to-Run*. Una vez realizado este proceso, se realizará un *Context-Switch* a la Task en cuestión. Si actualmente la Tarea en la que nos encontramos es la Task más prioritaria *Ready-to-Run*, nos saltaremos la parte de código encargada de realizar un *Context Switch*.

Recalcar de nuevo que la función es llamada a nivel de Task, y por tanto no puede ser empleada para hacer rescheduling al finalizar las ISRs. Para este último caso, se usará la función OSIntExit(), nombrada anteriormente.

Conceptos e implementación

Scheduling: Internals

```c
void  __attribute__ ((nomips16)) OSSched (void)
{
    CPU_SR_ALLOC();



    if (OSIntNestingCtr > (OS_NESTING_CTR)0) {
        return;
    }

    if (OSSchedLockNestingCtr > (OS_NESTING_CTR)0) {
        return;
    }

    CPU_INT_DIS();
    OSPrioHighRdy    = OS_PrioGetHighest();
    OSTCBHighRdyPtr = OSRdyList[OSPrioHighRdy].HeadPtr;
    if (OSTCBHighRdyPtr == OSTCBCurPtr) {
        CPU_INT_EN();
        return;
    }

#if OS_CFG_TASK_PROFILE_EN > 0u
    OSTCBHighRdyPtr->CtxSwCtr++;
#endif
    OSTaskCtxSwCtr++;

    OS_TASK_SW();
    CPU_INT_EN();
}
```

El código fuente de la función es el que se muestra en la figura. Antes de nada, se debe crear una variable local que será utilizada en el proceso de save/restore del status de interrupciones. Esta variable local (en uC/OS-III) es cpu_sr. La variable es declarada con la macro **CPU_SR_ALLOC**();

El proceso de guardado de los registros de status de interrupciones se debe a que vamos a entrar en una <u>sección crítica</u>, y por tanto se deben deshabilitar las interrupciones. El proceso de deshabilitarlas, implica que se debe guardar previamente el estado actual de las mismas (habilitaciones particulares de cada módulo con ISRs, CoreTimer, Interrupciones *Uart*, *SPI*, *PWM*, *Input Capture*, prioridades y subprioridades asignadas a las mismas…) . Una vez se realiza el proceso de *Context-Switch* (si fuese necesario), se restaura de nuevo el registro status de interrupciones del registro 12 del coprocesador 0 (Esto es en particular para el micro con el que trabajamos, pero el hecho de guardar/restaurar registros de estado de interrupciones es análogo para otras arquitecturas.)

```c
#define  CPU_SR_ALLOC()              CPU_SR  cpu_sr = (CPU_SR)0
```

```c
#define  CPU_INT_DIS()          do { cpu_sr = CPU_SR_Save(); } while (0)
#define  CPU_INT_EN()           do { CPU_SR_Restore(cpu_sr); } while (0)
```

Conceptos e implementación

Scheduling: Internals

```
#define  CPU_INT_DIS()        do { cpu_sr = CPU_SR_Save(); } while (0)
#define  CPU_INT_EN()         do { CPU_SR_Restore(cpu_sr); } while (0)
```

```
    .ent CPU_SR_Save
CPU_SR_Save:

    di     $2
    jr.hb $31
    nop

    .end CPU_SR_Save


    .ent CPU_SR_Restore
CPU_SR_Restore:

    mtc0  $4, $12, 0
    jr.hb $31
    nop

    .end CPU_SR_Restore
```

Conceptos e implementación

Scheduling: Internals

En caso positivo para realizar *Context-Switch* (debido a que la Task actual ya no es la más prioritaria en estado *Ready to Run*), se invoca a la función OS_TASK_SW(). Esta función es simplemente una macro que ejecuta una instrucción en ensamblador ("**syscall**").

```
#define  OS_TASK_SW()      asm volatile("syscall");
```

Esta instrucción va a generar una *exception* del sistema (_general_exception_context). A este vector de *exceptions* se encuentran ligadas otras funciones que pueden hacer uso de él. El código discrimina si la *exception* se ha generado por la llamada de *syscall* para proceder con el *Context-Switch*. En la figura de la derecha se muestra el código, el cual podemos localizar en el archivo **os_cpu_a.S**

```
    .ent _general_exception_context
_general_exception_context:
    saveCPU_CONTEXT                         /* Save CPU context to current Tasks Stack              */

    ori   $8,  $0,  0x007C                  /* Switch context only if a syscall exception occurred  */
    mfc0  $9,  $13, 0
    and   $9,  $9,  $8
    ori   $10, $0,  0x0020
    beq   $9,  $10, 0f
    nop

    la    $8,  _general_exception_handler   /* Call default_handler() for general exceptions not    */
    jalr  $8                                /* triggered by a SYSCALL. See Note 3                   */
    nop
    b     1f                                /* restore context after processing general exception   */
    nop

0:
    lw    $8,  STK_OFFSET_EPC($29)          /* Retrieve the stored EPC value from the Task's Stack  */
    addiu $8,  $8,  4                       /* EPC will reference the instruction following syscall */
    sw    $8,  STK_OFFSET_EPC($29)          /* See notes in comment header for explanation          */

    la    $8,  OSTCBCurPtr                  /* Save the current task's stack pointer                */
    lw    $9,  0($8)
    sw    $29, 0($9)                        /* OSTCBHighRdyPtr->StkPtr = SP;                        */

    la    $8,  OSTaskSwHook                 /* Call OSTaskSwHook()                                  */
    jalr  $8
    nop

    la    $8,  OSPrioHighRdy                /* Update the current priority.......                   */
    lbu   $9,  0($8)
    la    $10, OSPrioCur
    sb    $9,  0($10)                       /* OSPrioCur = OSPrioHighRdy                            */

    la    $8,  OSTCBHighRdyPtr              /* Update the current TCB.....                          */
    lw    $9,  0($8)
    la    $10, OSTCBCurPtr
    sw    $9,  0($10)                       /* OSTCBCurPtr = OSTCBHighRdyPtr                        */

    lw    $29, 0($9)                        /* Load the new task's stack pointer                    */
                                            /* SP = OSTCBHighRdyPtr->StkPtr;                        */
1:
    restoreCPU_CONTEXT                      /*Restore Task CPU environment                          */

    .end _general_exception_context
```

Conceptos e implementación

Scheduling: Internals

- **OSIntExit()** -> Llamada a nivel de ISRs

Al igual que OSSched(), OSIntExit() es una API propia del RTOS, encargada de comprobar y ejecutar una nueva planificación o scheduling. Sin embargo, esta función es llamada a nivel de ISRs (concretamente al finalizar el código situado en el Handler de la ISR). Esto se debe a que el Kernel necesita de un sistema para saber que se esta saliendo de una ISR (y que no se trata del flujo normal de funcionamiento), con el fin de comprobar si se ha producido el evento necesario para que una Task en particular (que estaba esperando por ese evento), pueda ejecutarse.

OSSched() es, por tanto, la función principal de scheduling llamada a nivel de Tasks para determinar si una nueva Task de mayor prioridad ha cambiado al estado *Ready-to-Run*. Una vez realizado este proceso, se realizará un *Context-Switch* a la Task en cuestión. Si actualmente la Tarea en la que nos encontramos es la Task más prioritaria *Ready-to-Run*, nos saltaremos la parte de código encargada de realizar un *Context-Switch*.

Recalcar de nuevo que la función es llamada a nivel de Task, y por tanto no puede ser empleada para hacer rescheduling al finalizar las ISRs. Para este último caso, se usará la función OSIntExit(), nombrada anteriormente.

Conceptos e implementación

Scheduling: Internals

```c
void  OSIntExit (void)
{
    CPU_SR_ALLOC();

    if (OSRunning != OS_STATE_OS_RUNNING) {
        return;
    }

    if (OSIntNestingCtr == (OS_NESTING_CTR)0) {
        return;
    }

    CPU_INT_DIS();
    OSIntNestingCtr--;
    if (OSIntNestingCtr > (OS_NESTING_CTR)0) {
        CPU_INT_EN();
        return;
    }

    if (OSSchedLockNestingCtr > (OS_NESTING_CTR)0) {
        CPU_INT_EN();
        return;
    }

    OSPrioHighRdy   = OS_PrioGetHighest();
    OSTCBHighRdyPtr = OSRdyList[OSPrioHighRdy].HeadPtr;
    if (OSTCBHighRdyPtr == OSTCBCurPtr) {
        CPU_INT_EN();
        return;
    }

#if OS_CFG_TASK_PROFILE_EN > 0u
    OSTCBHighRdyPtr->CtxSwCtr++;
#endif
    OSTaskCtxSwCtr++;

    OSIntCtxSw();
    CPU_INT_EN();
}
```

El código fuente de la función es el que se muestra en la figura. El código es muy similar al visto en la función OSSched(). Antes de nada, se debe crear una variable local que será utilizada en el proceso de *save/restore* del *status* de interrupciones. Esta variable local (en *uC/OS-III*) es cpu_sr. La variable es declarada con la macro **CPU_SR_ALLOC**();

Como diferencias fundamentales, vamos a tener control del número de interrupciones *anidadas* en las que se encuentra nuestra aplicación. Puesto que las ISRs forman la red más importante del sistema (recordad que la menor de las prioridades asignadas a una o varias ISRs siempre tiene más valor que la mayor de las prioridades que puedan asignarse a nivel de Tasks), no debemos realizar un *rescheduling* hasta que no hayamos llegado a un nivel de anidamiento igual a cero (dentro del mismo ciclo en el que se llama a OSIntExit()). En caso de que ***OSIntNestingCtr*** siga siendo > 0 después de haber eliminado un nivel de anidamiento, se debe volver a habilitar las ISRs, hasta lograr anidamiento cero. Cuando esta condición se cumple, procedemos a realizar el chequeo visto en *OSSched()* (determinación de la nueva Task *Ready to Run* de la *Ready List*, con el previo barrido de la *tabla de prioridades*)

Conceptos e implementación

Scheduling: Internals

```c
void  OSIntExit (void)
{
    CPU_SR_ALLOC();

    if (OSRunning != OS_STATE_OS_RUNNING) {
        return;
    }

    if (OSIntNestingCtr == (OS_NESTING_CTR)0) {
        return;
    }

    CPU_INT_DIS();
    OSIntNestingCtr--;
    if (OSIntNestingCtr > (OS_NESTING_CTR)0) {
        CPU_INT_EN();
        return;
    }

    if (OSSchedLockNestingCtr > (OS_NESTING_CTR)0) {
        CPU_INT_EN();
        return;
    }

    OSPrioHighRdy   = OS_PrioGetHighest();
    OSTCBHighRdyPtr = OSRdyList[OSPrioHighRdy].HeadPtr;
    if (OSTCBHighRdyPtr == OSTCBCurPtr) {
        CPU_INT_EN();
        return;
    }

#if OS_CFG_TASK_PROFILE_EN > 0u
    OSTCBHighRdyPtr->CtxSwCtr++;
#endif
    OSTaskCtxSwCtr++;

    OSIntCtxSw();
    CPU_INT_EN();
}
```

Hasta este punto, el chequeo de la nueva Task *Ready to Run* es igual que para el proceso seguido para tratar de hacer *rescheduling* a nivel de Tasks. Sin embargo, el proceso para realizar el *Context-Switch* es diferente (aunque obviamente muy similar al llamar a OS_TASK_SW() en OSSched()).

En este caso, ahora disponemos de la función OSIntCtxSw(). Esta función la encontramos también en **os_cpu_a.S**

Conceptos e implementación

Scheduling: Internals

```
    .ent OSIntCtxSw
OSIntCtxSw:
    la    $8, OSTaskSwHook
    jalr  $8
    nop

    la    $8,  OSPrioHighRdy
    lbu   $9,  0($8)
    la    $10, OSPrioCur
    sb    $9,  0($10)


    la    $8,  OSTCBHighRdyPtr
    lw    $9,  0($8)
    la    $10, OSTCBCurPtr
    sw    $9,  0($10)

    lw    $29, 0($9)


    restoreCPU_CONTEXT

    .end OSIntCtxSw
```

En la figura se muestra el código assembler de la función OSIntCtxSw().

Revisar Videoconferencia para ver explicación